

Hypothesis Space Reduction Using Uncertainty from Feature Pruning in Grid Load Prediction

Advanced Machine Learning Semester Project

Merlijn Mulder (s5187540)¹, Hinke Bolt (s4773454)¹, Luca Colli (s5047153)², and Martijn Schippers (s4556682)¹

¹Department of Artificial Intelligence, University of Groningen, 9747 AG Groningen, The Netherlands

²Department of Computing Science, University of Groningen, 9747 AG Groningen, The Netherlands

Abstract

When a model provides uncertainty estimates alongside its predictions, users can better assess the reliability of those predictions. This can be valuable in problems like grid load forecasting, where anticipating potential problems in advance is crucial. In such predictive tasks, explainable AI (XAI) techniques can guide feature selection by identifying irrelevant inputs. This can increase model performance. In this work, we demonstrate that feature reduction using XAI can also decrease aleatoric uncertainty, while epistemic uncertainty remains largely unaffected. Nonetheless, we observe fluctuations in both types of uncertainty. We employ a dynamic model whose hypothesis space scales with the input dimensionality ensuring the smallest effective model is used. Based on these findings, we propose an algorithm that jointly optimizes feature selection, model size, performance and uncertainty through XAI. The code implementation can be found at <https://github.com/MerlijnQ/AML>.

Introduction

The news of the biggest blackout in Europe in over 20 years in the Iberian Peninsula shocked the continent (Ambrose, 2025). Whilst European power grids are some of the most stable in the world, no grid is flaw-

less. Similarly, outages occur on other continents as well, as demonstrated by a failure in Panama earlier this year (Tico Times, 2025).

Identifying power requirement and consumption through energy load prediction in a smart grid can anticipate issues and prevent outages (Muralitharan et al., 2018). To achieve this, machine learning techniques can be applied and specialized into four different segments: very short-term forecasting (predict within a single day), short term forecasting (predict within weeks), medium-term forecasting (predict within months) and long-term forecasting (predict within years). This paper focuses on predictions in a 24-hour prediction time frame, as we argue this provides reasonable time to anticipate varying degrees of issues.

Often a Convolutional Neural Network (CNN) is used as the basis of this and has shown to be able to outperform networks based on long short-term memory (LSTM) for energy grid load prediction (Ibrahim & Rabelo, 2021). In order to prevent such models from overfitting and evaluate the model robustness, proposals have been made to insert adversarial noise into the training data when training a CNN (You et al., 2018). Simultaneously, training optimization techniques such as genetic algorithms (Muralitharan et al., 2018) and gradient boosting trees (Aguilar Madrid & Antonio, 2021) have shown to reduce the prediction error in energy grid load forecasting.

In these studies, the focus is often on the model

performance, while ignoring the uncertainty of the prediction. In machine learning, when we train a model on data, we induce a general hypothesis from this data, guided by empirical risk, on which we can make new predictions. In the case of a deep neural network, its size represents the hypothesis space which captures this general pattern in the data. Predictive uncertainty is inherent to this process of inductive inference, which can be divided into two sub-categories (Waegeman, 2021). These are aleatoric uncertainty, which is inherent to the noise in the data and therefore the variability in its outcome, and epistemic uncertainty, produced by the lack of knowledge of the true distribution.

Providing inference with predictive uncertainty informs the user how reliable the prediction is. For example, in a smart grid, we predict the energy load and let the generators only generate the predicted needed amount of electricity to save costs. When the uncertainty for that prediction is high, this can mean that there is a higher chance of possibly needing more electricity than predicted to prevent grid issues.

If we dynamically shrink the hypothesis space by removing the least essential information in an effort to reduce the chance of overfitting and to handle the curse of dimensionality this can either lead to

1. an increase in epistemic uncertainty, as we remove knowledge, or
2. a decrease in aleatoric uncertainty as non-essential features can be a source of noise in our hypothesis space.

In the context of the energy forecasting problem, aleatoric uncertainty within features come from, among others, variances in the unpredictable weather data, leading to discrepancies with actual predictions (Hasan et al., 2025). Likewise, traditional meters produce noisier data points than the smart meters (Hasan et al., 2025). Due to these uncertainties, some features can harm the prediction more than contribute to it.

Common features selected for this prediction task include time and weather based variables. For example, grid load is influenced by seasonality changes throughout the year and temporal features such as

the time of the day, making these features important for the task (Zabin et al., 2024). This is in line with Baur et al. (2024). Additionally, weather-based variables influence electricity generation, especially renewable energy, and could also indicate energy demand. These include air temperature, humidity, wind, rain, and cloud cover. Both categories of variables can indicate grid load, but could also indicate grid demand Baur et al., 2024.

Previous research (Aguilar Madrid & Antonio, 2021; Ibrahim & Rabelo, 2021) used statistical methods for feature selection. On the other hand, van Zyl et al. (2024) found that the use of explainable AI (XAI) methods could help in selecting the best features. It was found that when reducing the number of features, the accuracy improves. Furthermore, XAI allows analyzing why certain decisions are made, which can be beneficial compared to traditional methods. For example it prevents choosing a feature because in the training set this is predominant over the others.

van Zyl et al. (2024) examined two different model architectures for feature selection; namely, Gradient-weighted Class Activation Mapping (Grad-CAM) and Shapley Additive Explanations (SHAP). The results provided by Grad-CAM are less consistent than the results provided by SHAP, but the latter requires considerably more resources.

Previous studies used feature selection with the focus solely on improving the model performance (Aguilar Madrid & Antonio, 2021; Ibrahim & Rabelo, 2021). However, these studies did not evaluate the uncertainty of these predictions in these models. Therefore, we propose an algorithm that uses feature pruning to recursively discard features and reduce the network depth, while minimizing both the aleatoric and epistemic uncertainty, and the prediction error. We hypothesize that a smaller network, due to removing non-essential features, will result in higher accuracy and lower aleatoric uncertainty. At the same time, the epistemic uncertainty can increase with a smaller model due to a potential loss of knowledge.

For evaluating and testing this algorithm, we will use the grid load forecasting problem as context, due to its high number of possible influential features (making feature pruning possible) and chaotic nature

making it suitable for deep neural networks. Nevertheless, the proposed algorithm will be applicable to new problems for which feature importance is not known.

Methods

The following section describes details of the dataset and the process of preprocessing the data, building a model, selecting features by SHAP and the pipeline connecting all these components together.

The dataset

For our prediction task, we use a grid load dataset containing hourly data from 3rd of January 2015 until 27th of June 2020 (Aguilar Madrid, n.d.). This includes 48048 hours of data with 16 features, including historical electricity load, weather variables, and calendar variables, like holiday and school.

Data preprocessing

As Covid-19 occurred in the last year of the data, with a lockdown on the 25th of March 2020 (Aguilar Madrid & Antonio, 2021), we visually examined the data to investigate whether this pandemic or lockdown had an effect on the electricity grid load. As there was a decrease in grid load, as seen in Figure 1, all data after the 1st of March 2020 was discarded, leaving 45216 hours of the data.

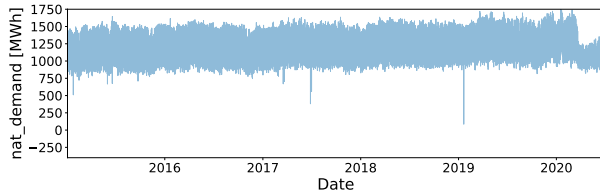


Figure 1: The grid load (in MWh) over the period from 3rd of January 2015 until 27th of June 2020. From the month March 2020 (the time Covid-19 started), a decrease in grid load can be observed compared to the overall trend.

The next step in the preprocessing of data, was to investigate if there is a yearly trend, seasonality and how much they contributed to the grid load. The yearly trend could be used to observe how the grid load varies over the years. The seasonality could indicate how the grid load demand fluctuates within a year and the residual load could indicate how much 'noise' is added. The following formula was used:

$$D_t = T_t + S_t + R_t \quad (1)$$

Where

D_t Grid load demand at time t

T_t Trend component at time t

S_t Seasonal component at time t

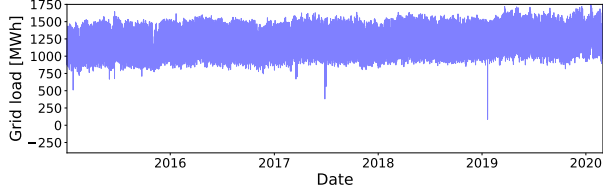
R_t Residual component at time t

The trend component was calculated by taking the simple moving average of the grid load over a 4-week period. The seasonal component was calculated by taking the simple moving average of the grid load minus the trend. The residual component was calculated by taking the grid load minus the trend and minus the residual components.

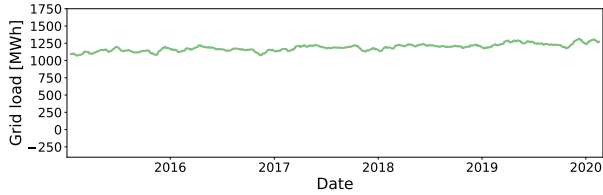
The results of the seasonality reduction can be seen in Figure 2. Although it was observed that there is a slight upward trend over the years from approximately 1100 MWh to 1300 MWh (see Figure 2b), this feature *year* was not included for prediction, as will be explained later in this section. Moreover, the seasonality component was not included as a feature, because the differences in load between months were negligible (see Figure 2c). From the residual load (see Figure 2d), it can be observed that there is still a noise, showing that there is unexplained variation in the data.

The feature *Holiday ID* has a possible explanatory value to the model, as the grid load for the Holiday IDs differ between IDs and differ compared to no holiday (see Figure 3). Therefore, this feature was hot encoded, adding 22 features to the set of features.

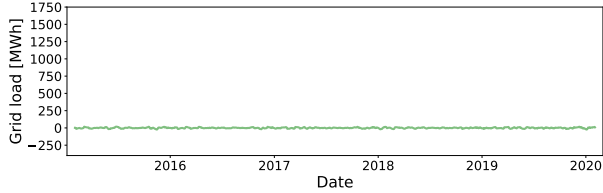
Besides the features in the dataset, we added other explainable features to the dataset: the features



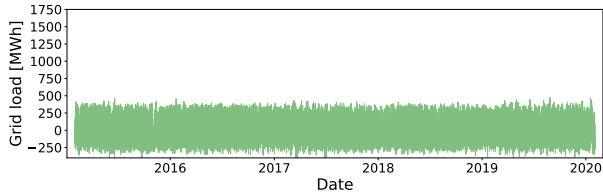
a: Observed grid load



b: Trend component



c: Seasonality



d: Residual load

Figure 2: Seasonal decomposition of Panama’s national grid load from 2015-01-03 to 2020-03-01. It displays (a) the observed total grid load, (b) the grid load trend over the years, (c) the seasonality of the data, and (d) residual load is what is left from the data when the trend and seasonality are deducted. Over the years, the electricity grid load has increased, the seasonality is not significant and there is still a significant amount of residual load.

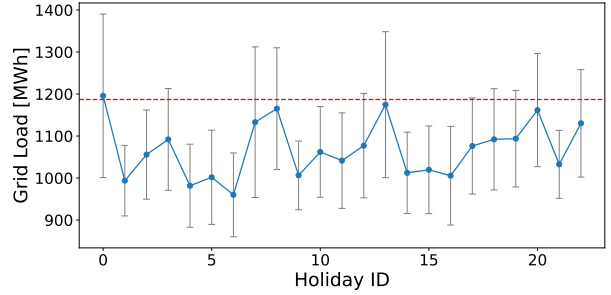


Figure 3: The grid load (in MWh) averaged for each Holiday ID, where ID=0 is no holiday. Overall, the grid load for the different holidays are lower than the average grid load.

weekend and *hour*. We visually examined Figure 4 and Figure 5, which showed, respectively, the relevance of the weekend and hour to the grid load. Therefore, we added the features *hour* and *weekend* by extracting them from the feature *datetime*. As the hour is a cyclical feature, it was encoded as the sine and cosine feature of the hour h as seen in equation 2 and 3, respectively.

$$\text{hour_sin} = \sin\left(\frac{h \cdot 2\pi}{23}\right) \quad (2)$$

$$\text{hour_cos} = \cos\left(\frac{h \cdot 2\pi}{23}\right) \quad (3)$$

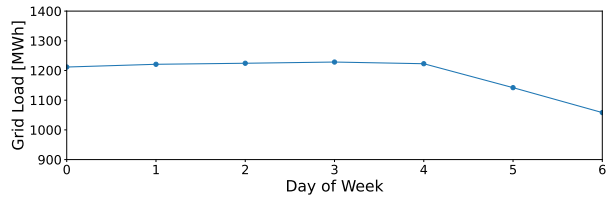


Figure 4: The grid load (in MWh) averaged per day of the week. The weekend (day 5 and 6) have a lower grid load compared to the other days.

In total, 40 features were used for prediction. All these features can be found in Appendix A.

Each data point’s input consists of $\mathbb{R}^n$ features and an input window of $\mathbb{R}^t$ hours. This results in a input

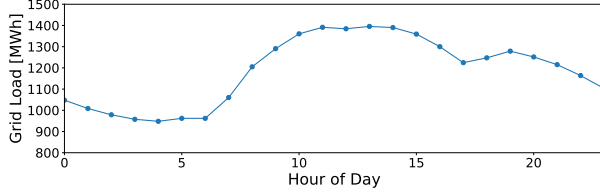


Figure 5: The grid load (in MWh) averaged for each hour of the day. It can be observed that the grid load is highest for the hours 11 until 14, while the lowest load is between the hours 3 to 6.

matrix with dimension: $d = \mathbb{R}^{n \times t}$. The prediction target is the grid load 24 hours in the future (t_{i+24}). Consequently, the number of data points equals the total number of hours in the dataset ($=45216$) minus the 24 hour forecast horizon and the input window length.

To observe the effect of historical data on the feature importance and model performance, the input window lengths of 24, 48, 72, and 120 hours were used.

Given the dataset, we sequentially split the data in a train, validation and test set, preventing leakage from test/validation into the train dataset. Given the sparsity of some features (i.e. the Holiday IDs), both the validation and test set consist of 1 year of data points. Therefore, the validation dataset has 8760 data points and the test dataset has 8784. The train dataset has 27648 minus the input window length.

As was previously mentioned, the *year* feature was not included in the model. Since the validation and test dataset only cover 1 year of the data, the *year* feature will not capture the overall trend (see Figure 2b).

After the train-validation-test split, we normalized the continuous features using the z-score (see equation 4). This was done by fitting to the train set and then transforming the train, validation and test set. To reduce instability when training the model, we also normalized the target feature due to its high target values with great variances.

$$\mathbf{z} = \frac{x - \mu}{\sigma} \quad (4)$$

Ultimately, for all the datasets, a separate dataloader was initialized with batch size 128.

Model

Layer definitions

The problem consists of a multivariate timeseries containing $\mathbb{R}^n$ input features and $\mathbb{R}^t$ timesteps as our input window providing an input dimension of $d = \mathbb{R}^{n \times t}$. Our time-axis has a sequential order t_{i+step} encoding the temporal patterns of the features over time. We can extract this by using a convolutional layer, that extracts these patterns into C_{out} channels. This can be defined as a cross-correlation in line with the PyTorch documentation (PyTorch Contributors, 2024) as seen in Equation 5. We apply a kernel with size K to an input with k channels at position t , moving over the input with a stride s , which is the starting position of the kernel and dilation α . We apply kernel element s to the input with a *dilation* to increase the receptive field of the kernel. This results in j new output channels for the n input elements.

$$\text{output}_{n,j}(t) = b_j + \sum_{k=0}^{C_{in}-1} \sum_{u=0}^{K-1} W_{j,k,s} \cdot \text{input}_{n,k}(t \cdot s + u \cdot \alpha) \quad (5)$$

Bai et al. (2018) proposed to use such convolutional layers in a residual block which improved performance on timeseries tasks. We build such a block by feeding an input x to Equation 8 which uses Equation 5, 6 (this decouples the magnitude of W from its direction) and 7 providing $\hat{x}$. By feeding $\hat{x}$ to equation to a ReLu activation as seen in Equation 8, we obtain $\tilde{y}_1$. This is then fed through a dropout layer with a dropout probability of 20% for regularization. This means that some parts of $\tilde{y}_1$ are set to zero. Simultaneously x is fed to 9, which output size depends on the size of $\tilde{y}_1$ resulting in $\tilde{y}_2$. We then compute $\tilde{z} = \tilde{y}_1 + \tilde{y}_2$ as the output of our residual block which we pass through a ReLu activation as seen in Equation 7. In order to keep the samples of the same timeseries length, we need to apply padding. As we want to perform causal convolutions, we only do this

add the left hand side. Therefore, the output at a specific timestep is only convolved with this and earlier time steps in the previous layers. This means that there is no data leakage from the future data in the convolutions.

$$W_{norm} = g \cdot \frac{v}{||v||} \quad (6)$$

$$\text{ReLU}(z) = \max(0, z) \quad (7)$$

$$y_{b,c,t} = \text{ReLU}\left(W_{norm}([\text{Conv1D}(x; W, b)]_{b,c,t})\right) \quad (8)$$

$$\tilde{x} = \begin{cases} x, & k = j \\ \text{Conv1D}_{1 \times 1}(x; W_p, b_p), & k \neq j \end{cases} \quad (9)$$

To extract the most important information from the timeseries, we define a global average-pooling layer as seen in equation 10. This will collapse the time dimension and represent all filters and timesteps in one final representation, which is fed to the two output heads.

$$\mathbf{z} = \frac{1}{T} \sum_{t=1}^T \mathbf{Z}_{:,t} \quad (10)$$

Lastly, we define a dense linear layer needed for a linear transformation of the input to calculate the final prediction as seen in equation 11. As we will work with batched operations, we can rewrite this as seen in equation 12.

$$\text{output}_i = b_i + \sum_{j=0}^{N_{in}-1} W_{i,j} \cdot \text{input}_j \quad (11)$$

$$\mathbf{y} = \mathbf{XW}^T + \mathbf{b} \quad (12)$$

Instead of predicting the variance σ^2 , we predict σ using the output head and square it in the loss term and the final prediction. This is to prevent σ^2 from exploding. Furthermore, σ has the property that it is always positive. Therefore, we define a linear layer

predicting ρ which gets trained by the network. We feed the output through

$$\text{Softplus}(x) = \log(1 + e^x)$$

to ensure positivity from which we obtain σ .

We initialize the weights in the variance head using a normal distribution with mean=0 and std= 10^{-4} and a bias where softplus(b)=1, to prevent unstable training due to high values.

Hypothesis Space Initialization

As seen in equation 5, the size of the convolutional layer depends on k , which is the number of input channels. We determine the number of filters by using geometric scaling. Here the first channel $c_0 = \text{int}(\min(8, \text{round}(4 * \sqrt{C_{k_0}})))$. The subsequent channels are determined by a growth g where $c_k = C_0 * g^b$, where b is the convolutional block in the network. We clamp the maximum channel size at 256, to prevent an overly big model with a high number of convolutional blocks. Furthermore, C_k is always rounded to the closest multiple of 8. This allows to use tensor cores which are activated when certain parameters of a layer are divisible by 8 (Sarge et al., 2019), optimizing compute.

Simultaneously, we let the input window $\mathbb{R}^t$ determine the depth of the network. We define the receptive field of B residual blocks as seen in Equation 13, where we define the dilation as $d_b = \alpha^b$ per convolutional block growing according with an exponential pattern where dilation $\alpha = 2$ happens in all layers except the residual convolutional layer in the block. Given that we want the size of the models receptive field $r_0 \geq N_t$ where N_t is the number of timesteps, we can calculate number of residual blocks L using equation 14. The full derivation is available in Appendix B.

$$r_0 = 1 + \sum_{b=0}^{B-1} 2\alpha^b (k-1) \prod_{j=0}^i s_j \quad (13)$$

$$B \geq \lceil \log_2\left(\frac{N_t - 1}{2(k-1)} + 1\right) \rceil \quad (14)$$

Uncertainty Estimation

As established before, the total predictive uncertainty is made up of the aleatoric (noise from the data) and epistemic (lack of information) uncertainty. In line with the information-theoretic approach, we estimate:

$$\sigma_{\text{total}}^2(x) \approx \sigma_{\text{epi}}^2(x) + \sigma_{\text{alea}}^2(x).$$

We use a two-headed model with an output head for the variance and an output head for the prediction. The variance will estimate the aleatoric uncertainty for a prediction, which is averaged over M forward passes:

$$\sigma_{\text{alea}}^2(x) \approx \frac{1}{M} \sum_{m=1}^M \sigma^2(x)^{(m)}$$

Simultaneously, the epistemic uncertainty will be estimated through the mean squared distance to the mean prediction over M forward passes:

$$\hat{\mu}(x) \approx \frac{1}{M} \sum_{m=1}^M \mu(x)^{(m)}$$

$$\sigma_{\text{epi}}^2(x) \approx \frac{1}{M} \sum_{m=1}^S (\mu(x)^{(m)} - \hat{\mu}(x))^2$$

We use the Bayesian inference framework to approximate

$$P(W \mid \mathcal{D})$$

by modeling

$$P(y \mid x)$$

for seven individual models with an identical architecture but different initializations, as this showed to be an optimal method for uncertainty estimation and disentanglement (Valdenegro-Toro & Saromo, 2022). Therefore, our entire ensemble approximates the distribution of possible weights

$$P(y \mid x, W)$$

. As we do not have direct access to $\mathcal{D}$, we model this using variational inference with different W as

seen in equation 15. This results in a posterior of the possible y and σ^2 used to estimate the uncertainties.

$$P(y \mid x, \mathcal{D}) \approx \frac{1}{M} \sum_{m=1}^M P(y \mid x, W^{(m)}) \quad (15)$$

Architecture

1. We apply L residual blocks with padding when using dilation to preserve the input size and a kernel size of 3 in all layers except the residual convolutional layer where the kernel size is 1.
2. We apply global pooling to reduce the channels to one vector representation
3. Finally, the vectors are fed to two separate output heads predicting the target value and the variance.
4. The final predictions from all ensemble members are aggregated, providing the final model prediction and uncertainty.

Loss

We are dealing with an optimization problem where we have two output heads, namely the mean and the variance. A common loss used in regression is the mean squared error, but this does not support an extra variance head. The Gaussian Negative Log Likelihood loss (GNLL) does do this, as seen in Equation 16. As we are training on both criteria at the same time, this can make training unstable if one overshoots. Therefore, we propose a warm-up phase using the mean squared error (MSE) as seen in Equation 17. This makes sure we first have a stable prediction head before training the variance head on top of this. We do this using a linear annealing schedule where

$$\alpha = \min(1, \text{epoch}/\text{warm_up})$$

and

$$\text{warm_up} = 10$$

. This results in the total loss seen in Equation 18.

$$\mathcal{L}_{GNLL} = \frac{1}{N} \cdot \sum_N^{i=0} \left(\frac{1}{2} \cdot \log(2\pi\sigma^2) + \frac{(y_i - \hat{y}_i)^2}{2\sigma^2} \right) \quad (16)$$

$$\mathcal{L}_{MSE} = \frac{1}{N} \cdot \sum_N^{i=0} (y_i - \hat{y}_i)^2 \quad (17)$$

$$\mathcal{L}_{total} = \alpha \cdot \mathcal{L}_{GNLL} + (1 - \alpha) \mathcal{L}_{MSE} \quad (18)$$

After the warm-up phase (10 epochs) is completed, the model will train using only $\mathcal{L}_{GNLL}$. In this phase, we make use of early stopping with a patience of 5 epochs to prevent overfitting. We assess the model performance using the validation loss. When this does not improve within the specified patience limit we stop training. Every time the loss decreases the model weights are saved in order to return the best performing model after training. This allows us to specify a high number of maximum epochs such as 100, in which early stopping will be triggered. Therefore the model only trains for the number of epochs it actually needs without overfitting to the training data.

SHAP for feature selection

As previously stated, although SHAP is more computationally expensive than gradient-based visualization methods such as Grad-CAM, the former is often able to provide more consistent feature attributions. Given that our study focuses on obtaining reliable explanations of the importance of each feature rather than real-time feature selection, the more consistent results of SHAP outweigh its higher computational costs. For these reasons, we chose SHAP as our explainable feature selection method.

SHAP is a machine learning method based on cooperative game theory that assigns Shapley values to each feature to quantify how much they contribute to the overall prediction (Lundberg & Lee, 2017). The traditional techniques for Shapley value estimation would require evaluating the model for all possible combinations of subsets of features, formally defined

by the equation

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} \cdot [f_{S \cup \{i\}}(x_{S \cup \{i\}}) - f_S(x_S)], \quad (19)$$

where F is the set containing all features and S is each possible subset excluding feature i . The number of all possible subsets of a set of size n is given by 2^n , which means that with a feature space of 40 features, calculating exact Shapley values would require $2^{40} \simeq 10^{12}$ evaluations of the model.

Computing the exact Shapley value using the original formula would require too many computational resources; for this reason, we decided to opt for a SHAP gradient explainer, which uses an extension of the integrated gradients method (Sundararajan et al., 2017) and provides an approximation of Shapley values that quantify the positive or negative contributions of each feature. This is done by taking a smaller background sample from the dataset to represent the expected behavior of the model. Integrated gradients are calculated with the formula

$$IG_i(x) = (x_i - x'_i) \int_{\alpha=0}^1 \frac{\partial f(x' + \alpha \cdot (x - x'))}{\partial x_i} d\alpha, \quad (20)$$

where x is the instance being explained, x' is the baseline input sampled from the background dataset, and f is the prediction function of the model. This means that the computational complexity of this SHAP explainer is given by $M \cdot B \cdot K$, with M representing the number of x instances being explained, B the number of x' background samples, and K the number of integration steps. Even though previous research suggests that larger background samples can produce more reliable explanations (Yuan et al., 2023), we chose $M=10$, $B=100$ in order to reduce the computational complexity of the model. The documentation of SHAP does not explicitly state how many integration steps were used, but based on the ranges suggested by Sundararajan et al. (2017), we might assume that the value of K is in the order of 100. The total number of iterations required to compute SHAP values with this method are approximately

$10 \cdot 100 \cdot 100 = 10^5$, or 10^7 fewer compared to the calculation of exact Shapley values.

Given that in the preprocessing, the hour feature was split into two separate features, the SHAP values for the sine and the cosine of the hour have been calculated independently. However, these features need to be combined in order to add meaning to the model. Therefore, their SHAP values have been averaged, and when performing feature selection they have been considered as one individual feature.

Pipeline

Our pipeline follows the procedure of iteratively initializing a model with a smaller feature dimension shown in Algorithm 1. The `DataLoader` class contains functions to retrieve samples from the training, validation and test sets with a batch size of 128. Uncertainty and accuracy metrics (RMSE), together with the discarded features and the SHAP values are recorded, using the test set, in arrays to allow saving them to a file and plotting figures at the end. The window sizes used are 24, 48, 72 and 120 hours. For each window size, a new model is trained, metrics are recorded in the arrays, and SHAP is used to remove the feature with the lowest SHAP value from the dataset. New smaller models are iteratively trained until no features are left.

Furthermore, we use spearman to test for a correlation between the measured variables and the number of features used to train the model. As the latter determines the model size this also relates to the size of the model. Lastly, we concatenate the data over all time windows to conclude on a correlation between the RMSE and the uncertainties using a linear mixed effects model.

Results

We tested the importance of features in 39 models, where a subsequent model excludes the least important feature used by the previous model. The importance was established using SHAP values, where the results can be observed in Figure 6, for an input window of 24-hour. The figure shows that the

Algorithm 1 The pipeline

```

1: Uncertainty  $\leftarrow \{\}$ 
2: Accuracy  $\leftarrow \{\}$ 
3: Discarded  $\leftarrow \{\}$ 
4: SHAP_values  $\leftarrow \{\}$ 
5: Explainer  $\leftarrow \text{SHAPExplainer}()$ 
6: for s in window_size do
7:   DataLoader  $\leftarrow \text{DataLoader}(s)$ 
8:   features  $\leftarrow \text{DataLoader.features}()$ 
9:   for n  $\leftarrow n_{\text{features}}$  down to 1 do
10:    model  $\leftarrow \text{Model}(n, s)$ 
11:    model  $\leftarrow \text{train}(\text{model}, \text{DataLoader})$ 
12:    Uncertainty[s, n]  $\leftarrow$  predict_uncertainty(model, DataLoader)
13:    Accuracy[s, n]  $\leftarrow$  evaluate(model, DataLoader)
14:    SHAP_values, Discarded[s, n]  $\leftarrow$  Explainer.explain(model, DataLoader)
15:    features  $\leftarrow \text{features} - \text{Discarded}[s, n]$ 
16:    DataLoader  $\leftarrow \text{DataLoader}(\text{features})$ 
17:  end for
18: end for
19: make_figure(Uncertainty, Accuracy, Discarded, SHAP_values)

```

nat_demand is the last feature to be discarded and always receives the highest SHAP value. The *holiday* and *school* features, together with all of the one hot encoded holidays, are the first ones to be removed as they were least important for the task prediction. The wind variables are, after the holiday features, the least important features, whereas the time and temperature have about the same importance. A similar pattern regarding the *holiday* features can be observed for different input windows, whereas the importance of the other features differ as seen in Appendix C.

The model performance with respect to the number of features included is seen in Figure 7. Taking the 24-hour window as an example, it can be observed that the model performs the worst when it includes the 15 most important features ($RMSE = 197.40 MWh$). The model performs best when it uses the 5 to 4 most important features ($RMSE =$

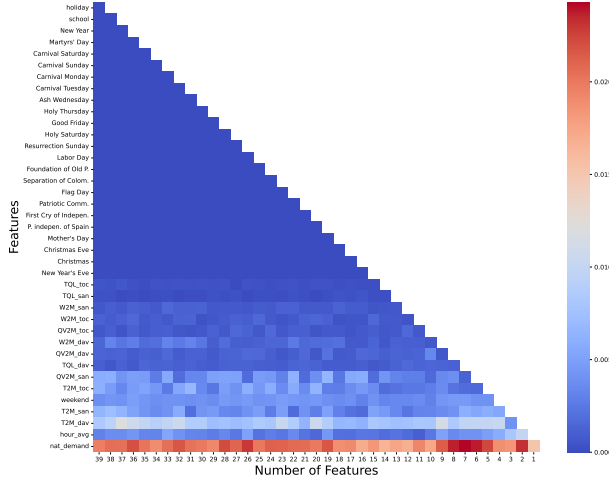


Figure 6: The figure illustrates importance of a feature for the model prediction through their SHAP values (higher is more important) with respect to the number of features included in the model. The vertical axis shows the included features, ranking from least to most important, and the horizontal axis show the number of features used for the prediction task. As displayed by the red color, *nat_demand* was the most important predictor for all models, whereas the holidays were least important for the prediction as indicated by the dark blue color.

196.60MWh). We find a significant monotonic correlation ($p < 0.05$) between the number of features and the RMSE for all input windows as seen in Table 1.

In Figure 8, we find that the aleatoric uncertainty is relatively the lowest for four features when including more than one feature ($\sigma_{24h} \approx 5.5, \sigma_{48h} \approx 6, \sigma_{72h} \approx 6, \sigma_{120h} \approx 7.5$). The uncertainty starts to decrease for all time windows when the number of features is about 11 and less. Before that, it fluctuates above 8 standard deviations. Notably, the aleatoric uncertainty is lower for the 24 hour time window, than for the other time windows. This suggests that some features add more noise when considering them over a longer time window. We find a roughly linear correlation ($p < 0.05$) between the number of features and the aleatoric uncertainty, where the uncertainty

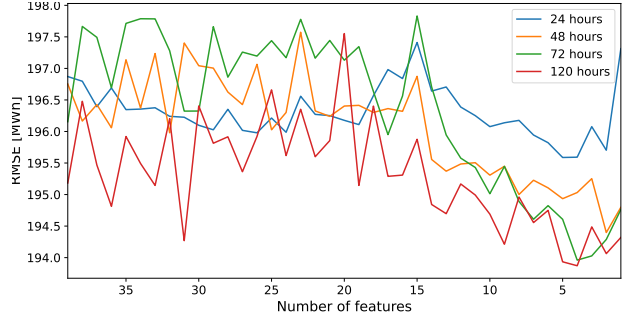


Figure 7: The graph illustrates the relation between the model performance, expressed through the RMSE in MWh, and the number of features included in the model, which determines the model size. We find the smallest error when using both the 5 and 4 most important features, except for a window size of 48 hours.

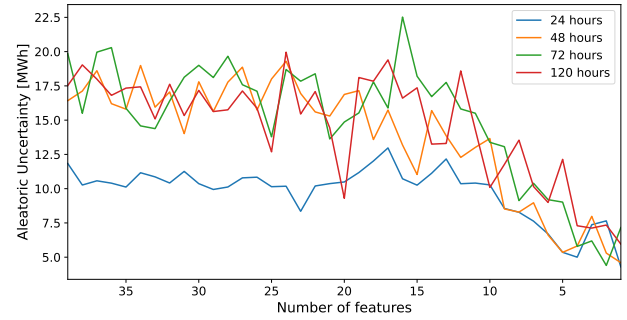


Figure 8: The graph illustrates aleatoric uncertainty in relation to the number of features used for the prediction for all input windows. The aleatoric uncertainty remains relatively high until it starts to decrease significantly when less than 10 features are included.

decreases with the number of features, for all input windows as seen in Table 1.

The epistemic uncertainty has the lowest value for a total number of 4 features when using 24-hour, 48-hour and 72-hour ($\approx 188.2MWh$), and 120-hour ($\approx 186.7MWh$) input windows, as seen in Figure 9. Furthermore, we can observe relatively large fluctuations in the graph. For all input windows, we do not

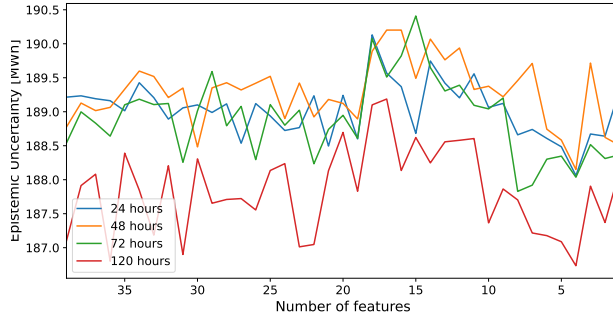


Figure 9: The graph illustrates the epistemic uncertainty in relation to the number of features used for the prediction for all input window hours. Between 11 and 4 features the uncertainty decreases until its lowest point. No stable pattern can be seen between 39 and 11 features.

conclude a significant correlation ($p > 0.05$) between the epistemic uncertainty and the number of features as seen in Table 1.

Window	RMSE	Aleatoric	Epistemic
24	($p = 0.044$, $\rho = 0.762$)	($p = 0.001$, $\rho = 0.502$)	($p = 0.160$, $\rho = 0.229$)
48	($p = 1.733 \cdot 10^{-8}$, $\rho = 0.324$)	($p = 9.463 \cdot 10^{-11}$, $\rho = 0.826$)	($p = 0.862$, $\rho = -0.029$)
72	($p = 3.689 \cdot 10^{-8}$, $\rho = 0.751$)	($p = 6.776 \cdot 10^{-6}$, $\rho = 0.625$)	($p = 0.231$, $\rho = 0.196$)
120	($p = 2.917 \cdot 10^{-5}$, $\rho = 0.617$)	($p = 5.114 \cdot 10^{-6}$, $\rho = 0.659$)	($p = 0.848$, $\rho = -0.032$)

Table 1: Spearman significance testing on the correlation between the number of input features and the measured variables per input window.

Using a linear mixed effects model, we establish that epistemic uncertainty is stronger correlated with RMSE ($p = 1.549 \cdot 10^{-3}$, $Coeff. = .343$) than aleatoric uncertainty ($p = 4.224 \cdot 10^{-16}$, $Coeff. = .125$).

Discussion

When looking at the RMSE for the 24, 72 and 120 hour windows in Figure 7, it can be seen that the RMSE values are the lowest when only four features are present, namely *nat_demand*, *hour_avg*, *T2M_san* and *T2M_dav* in all cases. Furthermore, the RMSE value for the 48 hour window is the lowest when only *hour_avg* and *nat_demand* are present. This suggests that a small subset of four features

might be sufficient to predict future energy load in Panama, which is coherent with our hypothesis that removing a certain number of features reduces the error in the model. This is also consistent with the findings of van Zyl et al. (2024), suggesting that explainable AI methods like SHAP can be useful in selecting which features are essential for the model, and which are redundant. In addition to improving the predictions, reducing the number of features makes the model more efficient and it reduces the time necessary for deployment.

One important advantage of SHAP over other dimensionality reduction methods such as Principal Component Analysis (PCA) is that the original features are not modified, and this allows getting more insight on the contribution of each feature to the model’s output.

Furthermore, it is shown that even though SHAP is particularly suitable for deterministic models, it can still be used to achieve reasonable results with stochastic models such as our ensemble. As suggested by Heese et al. (2023), using a large enough number of samples should allow reliable Shapley values for any model.

When analyzing the model performance and the uncertainties, we notice fluctuation suggesting that a stable decrease in error and uncertainty only happens after a certain threshold. Whilst the uncertainties are correlated with the RMSE, we notice that this does not provide for an identical pattern. This means that when the RMSE hits its lowest point with a certain number of features, when you discard another feature this can still give you a lower uncertainty whilst keeping the same model performance. Therefore, both variables have to be observed in order to select the best performing model with the lowest uncertainties.

We also observe that only aleatoric uncertainty is significantly correlated with the number of features in the model and not epistemic uncertainty. That the aleatoric uncertainty decreases with the number of features that we remove is in line with our hypothesis. This demonstrates that whilst some features in the dataset might seem important, like in this case the weather variables, they include too much noise to be able to make a reliable prediction with the proposed architecture.

Traditionally, the aleatoric uncertainty is said to be irreducible as it is inherent to the data. We argue that the aleatoric uncertainty that comes with a prediction can be minimized by removing the features from the model that are the most irrelevant. Because by removing the feature itself, you also remove its noise that makes the prediction less reliable.

Given what we established, we propose an algorithm that can be applied to the following general case. If you have a prediction task and you want to establish which features you should select for the model, you can establish this by testing multiple models that use different features. This should take into account the uncertainties to establish how reliable your prediction is. To do this in an efficient way, for each new model we can omit the least important feature based on explainable AI, to prevent testing a high number of models covering all possible combinations of input features. This requires less compute whilst being more explainable.

As we have seen in our results, due to fluctuations, we can end up in a local minima. Hence, when the error and uncertainties increase again, this does not mean we found the most optimal point. To prevent this, we should either test all models until their is one feature left or use early stopping with an appropriate patience to save compute. We then define a loss function that is a combination of the model error and the uncertainties, where the per variable importance can be tailored to the task $\mathcal{L}_{feature_pruning} = \beta \cdot RMSE + \gamma \cdot aleatoric + \delta \cdot epistemic$.

In summary, we propose a training algorithm that can be applied in the scenario described to find the most optimal model, including uncertainty, whilst explaining the importance of the features that are used as seen in Algorithm 2.

Future research

The model’s performance and uncertainty estimates rely on the capacity of the SHAP explainer to determine which features should be discarded. Due to resource limitations, relatively a small number of samples was used to determine the Shapley values. In this study, we only used 10 samples from the test dataset to determine the feature’s importance. Due

Algorithm 2 Proposed algorithm for model finding the optimal model

```

1: DataLoader  $\leftarrow$  DataLoader(all_features)
2: prev_loss  $\leftarrow$  inf
3: prev_model  $\leftarrow$  None
4: Discarded  $\leftarrow$  []
5: patience  $\leftarrow$  5
6: no_improve  $\leftarrow$  0
7: best_features  $\leftarrow$  []
8: for n  $\leftarrow$  n_features down to 1 do
9:   model  $\leftarrow$  Model(n, s)
10:  model  $\leftarrow$  train(model, DataLoader)
11:  Epistemic, aleatoric  $\leftarrow$  predict_uncertainty(model, DataLoader)
12:  RMSE  $\leftarrow$  evaluate(model, DataLoader)
13:  Discarded[n]  $\leftarrow$  shap(model, DataLoader)
14:  features  $\leftarrow$  all_features - Discarded
15:  DataLoader  $\leftarrow$  DataLoader(features)
    loss  $\leftarrow$   $\beta \cdot RMSE + \gamma \cdot aleatoric + \delta \cdot epistemic$ 
16:  if loss > prev_loss then
17:    if no_improve == patience then
18:      Return best_model, best_features
19:    end if
20:    no_improve  $\leftarrow$  no_improve + 1
21:  end if
22:  no_improve  $\leftarrow$  0
23:  best_features  $\leftarrow$  features
24:  prev_loss  $\leftarrow$  loss
25:  prev_model  $\leftarrow$  model
26: end for

```

to the sparsity of some features, like the Holiday IDs being used for only one day a year, the likelihood of that day to be included in a 10 samples is low. Therefore, our algorithm could have discarded them because in the provided data samples, the holidays were underrepresented or not represented at all.

Furthermore, the model performance relies on which features are included in the data. We did not include the trend component of the grid load, while we observed an upward trend. Therefore, not encoding the date as a feature could have resulted in a higher RMSE.

In addition, the calendar features (i.e. the holi-

day) included in the model represent the days within the input window, rather than indicating whether the time of the prediction itself is a holiday. Consequently, the predicted grid load may already reflect the average grid load at the time the holiday is in the input window, or the model may not know that the prediction time corresponds to a holiday. As a result, the contribution of the holiday features to the models may be smaller than expected, and may be discarded earlier by the SHAP explainer.

Our proposed algorithm reduces the model size based on the number of features. Therefore, it can only prune the model when there is a high number of features included in the dataset. Consequently, the applicability of our proposed algorithm to new problems decreases with the number of features available in the new dataset.

Future research should focus on investigating whether SHAP provides a reliable enough score when a feature is strongly underrepresented in the dataset. This might be done by using an entire year of samples to obtain accurate SHAP values for sparse one-hot encoded features and then comparing them to iterations with a lower number of samples.

Moreover, future research should focus on evaluating our proposed algorithm to new problems with different models (i.e. LSTM) and datasets. In doing so, it is important to ensure that the number of samples for the SHAP explainer is sufficient to get consistent explanations of the feature importance.

Contributions

The group used a mix of collegial and centralized system in order to allow everybody to contribute equally to the decisions making process, and, at the same time, being able to push the project towards a certain direction in case of disagreements. In general, no significant conflicts between the group members arose. To make the workload more efficient, we assigned a field of expertise to each group member, who was responsible for conducting research about the topic, writing the code and the corresponding section of the report. Whenever multiple group members were undecided about how to proceed about a certain matter,

the opinion of the group member who was assigned that specific field or the group leader was asked.

This is the specific distribution of the tasks:

- Merlijn: Group leader; designing and implementing the model
- Martijn: Creation of images and analysis of the results
- Hinke: Dataset and dataloader
- Luca: Feature selection with SHAP and pipeline

References

- Aguilar Madrid, E. (n.d.). Short-term electricity load forecasting (Panama). <https://www.kaggle.com/datasets/ernestojaguilar/shortterm-electricity-load-forecasting-panama/data>
- Aguilar Madrid, E., & Antonio, N. (2021). Short-term electricity load forecasting with machine learning [Publisher: Multidisciplinary Digital Publishing Institute]. *Information*, 12(2), 50. <https://doi.org/10.3390/info12020050>
- Ambrose, J. (2025, May). ‘blackouts can happen anywhere’: How power systems worldwide can collapse. <https://www.theguardian.com/business/2025/may/02/blackouts-energy-outage-risks-europe-worldwide-spain-portugal-france>
- Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. <https://arxiv.org/abs/1803.01271>
- Baur, L., Ditschuneit, K., Schambach, M., Kaymakci, C., Wollmann, T., & Sauer, A. (2024). Explainability and interpretability in electric load forecasting using machine learning techniques—a review. *Energy and AI*, 16, 100358.
- Hasan, M., Mifta, Z., Papiya, S. J., Roy, P., Dey, P., Salsabil, N. A., Chowdhury, N.-U.-R., & Farrok, O. (2025). A state-of-the-art comparative review of load forecasting methods: Characteristics, perspectives, and applications. *Energy Conversion and Management: X*, 26, 100922. <https://doi.org/10.1016/j.ecmx.2025.100922>

- Heese, R., Mücke, S., Jakobs, M., Gerlach, T., & Pitkowski, N. (2023). Shapley values with uncertain value functions. In B. Crémilleux, S. Hess, & S. Nijssen (Eds.), *Advances in intelligent data analysis xxi* (pp. 156–168). Springer Nature Switzerland.
- Ibrahim, B., & Rabelo, L. (2021). A deep learning approach for peak load forecasting: A case study on panama. *Energies*, 2029. <https://doi.org/10.3390/en14113039>
- Lundberg, S., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. <https://arxiv.org/abs/1705.07874>
- Muralitharan, K., Sakthivel, R., & Vishnuvarthan, R. (2018). Neural network based optimization approach for energy demand prediction in smart grid. *Neurocomputing*, 273, 199–208. <https://doi.org/10.1016/j.neucom.2017.08.017>
- PyTorch Contributors. (2024). *Pytorch documentation*. PyTorch. <https://pytorch.org/docs/stable/>
- Sarge, V., Andersch, M., Fabel, L., Micikevicius, P., & Tran, J. (2019). Tips for optimizing gpu performance using tensor cores. <https://developer.nvidia.com/blog/optimizing-gpu-performance-tensor-cores/>
- Sundararajan, M., Taly, A., & Yan, Q. (2017). Ax- iomatic attribution for deep networks. <https://arxiv.org/abs/1703.01365>
- Tico Times. (2025, March). *Nationwide power outage hits panama after generator failure*. <https://ticotimes.net/2025/03/16/nationwide-power-outage-hits-panama-after-generator-failure>
- Valdenegro-Toro, M., & Saromo, D. (2022). A deeper look into aleatoric and epistemic uncertainty dis- entanglement. <https://arxiv.org/abs/2204.09308>
- van Zyl, C., Ye, X., & Naidoo, R. (2024). Harnessing explainable artificial intelligence for feature selec- tion in time series energy forecasting: A compara- tive analysis of grad-cam and shap. *Applied Energy*, 353, 122079. <https://doi.org/10.1016/j.apenergy.2023.122079>
- Waegeman, E. H. W. (2021). Aleatoric and epistemic uncertainty in machine learning: An introduction to concepts and methods. *Mach Learn*, 547–506. <https://doi.org/10.1007/s10994-021-05946-3>
- You, Z., Ye, J., Li, K., Xu, Z., & Wang, P. (2018). Adversarial noise layer: Regularize neural network by adding noise. <https://arxiv.org/abs/1805.08000>
- Yuan, H., Liu, M., Kang, L., Miao, C., & Wu, Y. (2023). An empirical study of the effect of back- ground data size on the stability of shapley addi- tive explanations (shap) for deep learning models. <https://arxiv.org/abs/2204.11351>
- Zabin, R., Haque, K. F., & Abdelgawad, A. (2024). Predxgbr: A machine learning framework for short- term electrical load prediction. *Electronics*, 13(22), 4521.

A Features in the dataset

This appendix shows the different features from the dataset and included in the model (see Table A1. The feature *Holiday ID* was one hot encoded. Table A2 shows the IDs with their corresponding holidays.

Feature	Explanation	Included in model	Datatype
Datetime	Date-time index	No	
Nat_demand	National electricity load (MWh)	Yes	Continuous
T2M_toc	Temperature at 2 meters in Tocumen, Panama city	Yes	Continuous
QV2M_toc	Relative humidity at 2 meters in Tocumen, Panama city	Yes	Continuous
TQL_toc	Liquid precipitation in Tocumen, Panama city	Yes	Continuous
W2M_toc	Wind speed at 2 meters in Tocumen, Panama city	Yes	Continuous
T2M_san	Temperature at 2 meters in Santiago city	Yes	Continuous
QV2M_san	Relative humidity at 2 meters in Santiago city	Yes	Continuous
TQL_san	Liquid precipitation in Santiago city	Yes	Continuous
W2M_san	Wind speed at 2 meters in Santiago city	Yes	Continuous
T2M_dav	Temperature at 2 meters in David city	Yes	Continuous
QV2M_dav	Relative humidty at 2 meters in David city	Yes	Continuous
TQL_dav	Liquid precipitation in David city	Yes	Continuous
W2M_dav	Wind speed at 2 meters in David city	Yes	Continuous
Holiday_ID	Unique identification for different holidays (0-22)	One-hot encoded	Discrete
holiday	Holiday binary indicator (1=holiday, 0=regular day)	Yes	Discrete
school	School period binary indicator (1=school, 0=vacations)	Yes	Discrete
weekend	Weekend binary indicator (1=weekend, 0=weekday)	Yes	Discrete
hour	Hour in the day	Cyclical encoded	Discrete

Table A1: The features of the original dataset and those included after preprocessing. The features are given with their explanation, how they are included, and the datatype of the feature (continuous or discrete). The column how the feature is included in the model has the values No (not included), Yes (included with the current value), one hot encoded, or cyclical encoded (sine & cosine).

Holiday ID	Corresponding holiday
1	New Year
2	Martyrs' Day
3	Carnival Saturday
4	Carnival Sunday
5	Carnival Monday
6	Carnival Tuesday
7	Ash Wednesday
8	Holy Thursday
9	Good Friday
10	Holy Saturday
11	Resurrection Sunday
12	Labor Day
13	Foundation of Old Panama
14	Separation of Panama from Columbia
15	Flag Day
16	Patriotic Commemoration in Colon city
17	First Cry of Independence
18	Independence of Panama from Spain
19	Mother's Day
20	Christmas' Eve
21	Christmas
22	New Year's Eve

Table A2: The *Holiday ID* feature with its corresponding holiday.

B Receptive Field for B Residual Blocks

Given that we have the receptive field formula

$$r_0 = 1 + \sum_{l=0}^{L-1} ((k_l - 1) \prod_{i=0}^{L-1} s_i)$$

, we determine that our stride $s = 1$ for all layers we derive

$$r_0 = 1 + \sum_{l=0}^{L-1} (k_l - 1)$$

. This is our receptive field per layer, but we want to obtain it per residual block B , which includes 2 convolutional layers. If we keep our kernel size k constant for all layers we get

$$r_0 = 1 + \sum_{b=0}^{B-1} 2(k - 1)$$

Now we introduce a dilation α in our layers to increase the receptive field exponentially per block, changing r_0 to

$$r_0 = 1 + 2(k - 1) \cdot \sum_{b=0}^{B-1} \alpha^b$$

. From here, we obtain

$$r_0 = 1 + 2(k - 1) \cdot \frac{\alpha^B - 1}{\alpha - 1}$$

which we can simplify with a constant of $\alpha = 2$ to

$$r_0 = 1 + 2(k - 1) \cdot (2^B - 1)$$

. When we want the model to see our entire timeline for N_t timesteps, we need $N_t \leq r_0$. Therefore

$$N_t \leq 1 + 2(k - 1)(2^B - 1)$$

where after rearranging terms we get

$$2^B \geq \frac{N_t - 1}{2(k - 1)} + 1$$

. Finally we derive

$$B \geq \lceil \log_2 \left(\frac{N_t - 1}{2(k - 1)} + 1 \right) \rceil$$

, which determines the depth of our network.

C Heatmaps

Figure C1 shows the heatmaps for the different window sizes displaying the feature importance in relation to the number of features in each model. We observe that the feature *nat_demand* contributes the most to the models in each run. However, we also observe that the feature *nat_demand* contributes more to the models in the 24 hour window compared to the models of the other input windows. This indicates that the *nat_demand* a day before the hour of the target, it contributes more to the actual prediction than the *nat_demand* at times further away from the prediction.

Moreover, the order of features discarded from the models differs between the input windows. For example, the *hour* feature contributes the most to the models after the *nat_demand* feature, except in the 120 hour input window. Therefore, a bigger input window shows that the *hour* is less important in explaining the predictions than other features.

Furthermore, in all windows, we observe that most of the calendar features (the *holiday*, *school* and one hot encoded holiday features) are first to be discarded due to the lowest Shapley values. Only in the window sizes 72 and 120, there are instances a weather variable was discarded before the *holiday* calendar variable. This is likely because the larger the input window, the higher the chance a sparse *holiday* feature appears in the input of one of the 10 samples used by the SHAP explainer.

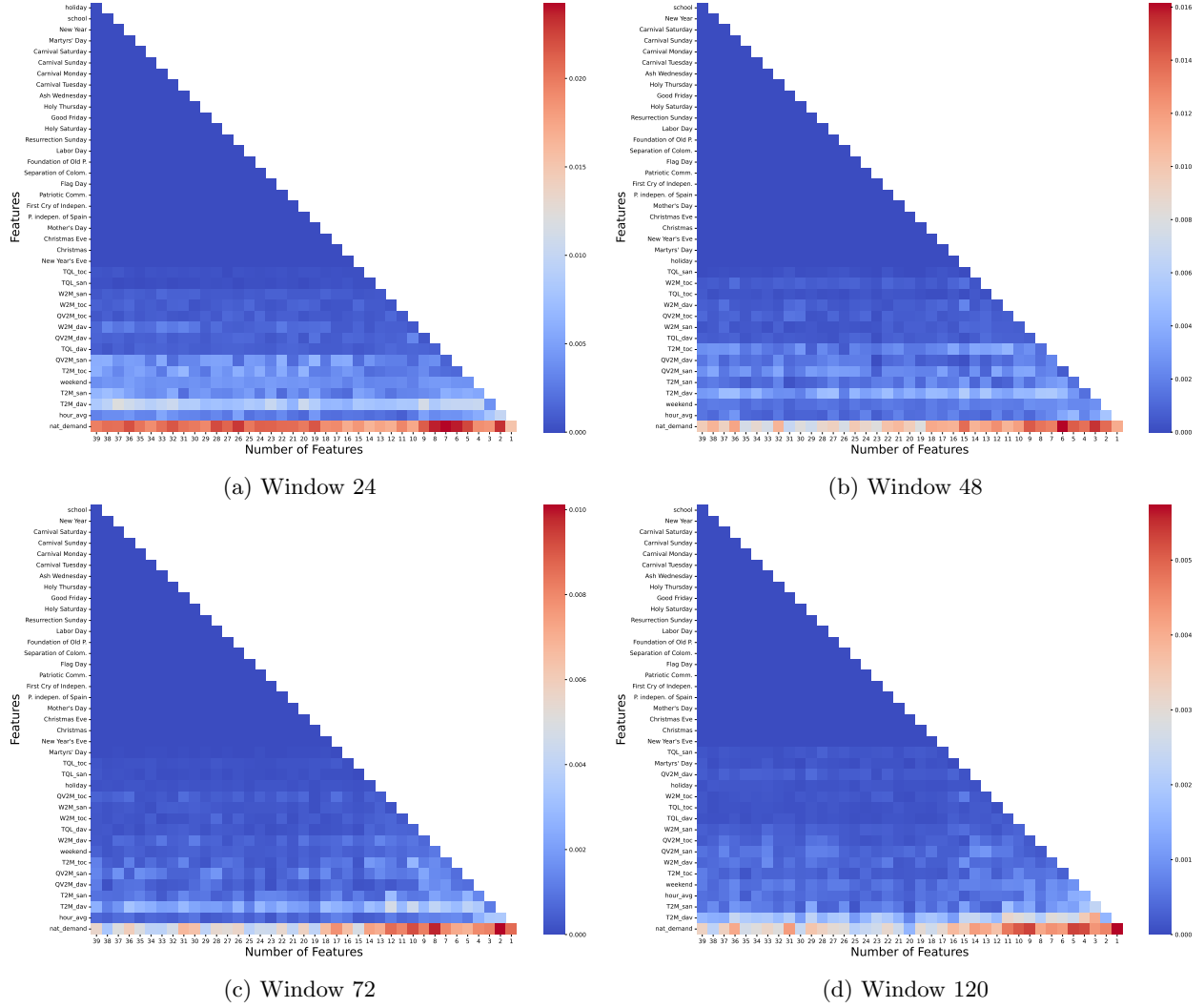


Figure C1: Comparison of heatmaps for the different window sizes (24, 48, 72, and 120 hour). What is shown in the heatmap, is the relation between the features and their importance in model prediction, as determined by SHAP. On the y-axis, all different features are displayed. On the x-axis, the different SHAP runs are displayed. After each SHAP run, the least important feature is discarded for the next SHAP run. The color of a block in the heatmap indicate the importance of a certain feature in a certain run. The more blue colors indicate a relatively small importance and the more red colors indicate a relatively big importance, as can be seen in the legenda. The white space of a feature/run combination indicate that that feature is discarded.